CYBERVIE.COM
A CYBER SECURITY FIRM

AI Engineer Bootcamp

Production GenAI, RAG & Agents

Course content for 45 live classes

“Don’t just use AI. Learn to engineer it.

The biggest opportunities of the AI era will belong to those who can turn intelligence into real-world solutions.”

- From transformer internals and a GPT you code yourself to deployed RAG systems and multi-agent applications
- Four portfolio projects and a capstone
- Career paths into AI engineering from any IT role

About This Course

> *"AI Engineering will become as foundational to the next generation of professionals as software skills became to the last. Those who learn it won't simply adapt to the AI revolution—they'll shape it."*

What is AI Engineering?

AI Engineering is the discipline of building, shipping and running real software products powered by AI models, mainly large language models (LLMs). ***An AI engineer usually does not invent new models. Instead, they understand how those models work from the inside (tokens, attention, training, inference cost) and use that knowledge to turn existing models from OpenAI, Anthropic, Google or the open-source community into reliable applications: assistants that answer from a company's own documents (RAG), agents that call tools and finish multi-step tasks, document-processing pipelines, and copilots embedded in products.***

The job sits between software engineering, data/ML and DevOps. It means writing clean backend code, connecting models to data and tools, measuring whether the AI is actually correct, keeping cost and latency under control, securing the system against misuse, and deploying it so it keeps working in production. In 2026 hiring data, "AI Engineer" has become the umbrella title for this work, and employers care most about what you have shipped.

Who is this AI Engineering course for?

- **Software developers** (backend, full-stack, mobile, QA, DevOps) who want to move into AI Engineer or Applied AI Engineer roles.
- **IT professionals from any field** (testing, infrastructure, data, support, enterprise platforms, business analysis) who want a structured move into AI roles. The Career Paths chapter maps each background to target roles.
- **Fresh graduates and career switchers** with working Python who want a job-ready portfolio rather than theory alone.
- **Data analysts, data scientists and ML practitioners** who work in notebooks and want to learn production deployment, evaluation and agents.
- **Technical leads and architects** who need hands-on command of RAG, agents and LLMOps to design and review AI systems.

- **Founders and product builders** who want to build and launch AI features themselves.

What will we learn in this AI Engineering course?

Over 45 live classes you will learn to:

1. **Understand models from the inside:** neural-network training on MNIST, tokenization, embeddings, attention (QKV, self-, cross- and multi-head), causal masking, and a GPT you code yourself. (Classes 1–8)
2. **Follow a model from training to production:** pretraining, post-training, distributed training, KV caching, quantization and LoRA. (Classes 9–10)
3. **Build LLM applications:** prompting, provider APIs, structured outputs, tool calling, cost and latency control, and FastAPI serving. (Classes 11–15)
4. **Build RAG systems end to end:** chunking, vector databases, hybrid search, reranking, query rewriting, GraphRAG, KAG, agentic and multimodal RAG, guardrails, and Code RAG. (Classes 16–23)
5. **Evaluate and monitor AI:** golden datasets, RAG-specific evals, LLM-as-judge, tracing, and eval gates in CI. (Classes 24–26)
6. **Build agents and context systems:** ReAct, Pydantic AI, LangChain, LangGraph, context engineering, memory, MCP, and multi-agent orchestration. (Classes 27–32)
7. **Work with reasoning and multimodal models, and customize them:** thinking models, CLIP, vision-language models, diffusion, prompting vs RAG vs fine-tuning, LoRA/QLoRA, and vLLM. (Classes 33–37)
8. **Deploy and operate in the cloud:** Docker, Kubernetes, AWS Bedrock with Azure and GCP compared, CI/CD, MLflow, and LLMOps. (Classes 38–40)
9. **Secure and govern AI systems:** prompt-injection defense, guardrails, PII handling, OWASP LLM Top 10, NIST AI RMF, ISO/IEC 42001, and the EU AI Act. (Class 41)
10. **Launch your career:** a deployed capstone, live demos, system-design interviews, a role map and a transition plan. (Classes 42–45)

Why This Course Looks the Way It Does

We reviewed live and recent postings on Dice and LinkedIn (through search listings, not scraping), plus hiring-trend reports built on tens of thousands of postings. Five patterns drove the design.

1. "AI Engineer" has become the umbrella title. Reports in 2026 show it absorbing the older "ML Engineer" title, with sibling roles around it: LLM Engineer, Applied AI Engineer, Agentic AI Engineer, AI Platform Engineer, Forward-Deployed Engineer, and MLOps/LLMOps Engineer.

2. The common stack is Python, LLM APIs, RAG, agents, cloud and observability. Repeated phrases in current listings:

Cluster	What postings say
Language / backend	Python (dominant), FastAPI, async programming, TypeScript/React for full-stack roles, Node.js and Java variants
LLM layer	OpenAI, Anthropic (Claude), Gemini; prompt engineering, structured outputs, function/tool calling
Retrieval	RAG, vector databases (Pinecone, Weaviate, pgvector, Neo4j), hybrid search
Orchestration	LangChain, LangGraph, LlamaIndex, CrewAI, multi-agent workflows, MCP
Evaluation / observability	LangSmith, Langfuse, ragas, Grafana, Dynatrace, Splunk
Cloud / platform	AWS Bedrock, Azure AI Foundry / Azure ML, GCP Vertex AI, Cloud Run, BigQuery
DevOps	Docker, Kubernetes, GitHub Actions, CI/CD, MLflow
Deep learning	PyTorch (more than TensorFlow), Hugging Face, fine-tuning
Governance	Responsible AI, security, guardrails (visible in bank and enterprise postings)

3. The numbers behind it (figures vary by source and method, so treat them as directional):

- Python appears in roughly 71% to 89% of AI postings depending on the study.
- RAG shows up in about 65% of applied-LLM listings.
- LLM fine-tuning appears in about 28% of AI engineering postings; PyTorch about 38%, TensorFlow about 33%.
- In MLOps postings: Python about 78%, cloud platforms about 69%, AWS named in more than half, MLflow just under 30%.
- Only around a quarter of AI postings require an AI-specific or advanced degree. Employers screen on shipped systems.
- Hiring managers repeat one complaint: many candidates can build in a notebook, few have deployed and operated something in production.
- Forward-deployed and customer-facing AI engineer postings grew sharply in 2025, which is why the course includes solution design and communication work.

4. Seniority bar. Many live Dice postings ask for 5+ years of software engineering plus 2+ years of AI/GenAI. Entry-level postings exist but expect strong software engineering. So tests, APIs, containers, CI and observability are taught alongside the AI topics, not after them.

5. Model internals are part of the job, not trivia. Cost, latency, context limits and failure behavior all trace back to how tokenization, attention, KV caches and quantization work. Classes 4–10 build these pieces from scratch

(attention, a GPT, KV caching), and later classes keep returning to them when debugging production behavior. This is a design decision, not a statistic from the postings.

What this means for the course

- Roughly **60%** of class time is hands-on labs.
- Every module ends with a **shippable artifact** that goes into the student's portfolio (live demo link plus GitHub repo).
- **Evaluation, observability and security** each get dedicated classes, since these separate production candidates from demo builders.
- Cloud is taught on **one primary platform in depth (AWS Bedrock)** with Azure AI Foundry and GCP Vertex AI compared, because postings split across all three.
- Labs are model-agnostic: they run against OpenAI, Anthropic and Gemini APIs plus open models, so students can say they have used each.

Skill employers ask for	Classes
Python, FastAPI, async	3, 15
LLM APIs, prompt engineering, structured outputs, tool calling	4, 11–14
Transformer internals, tokenization, embeddings, attention	4–8
Training at scale and inference optimization (KV cache, quantization, LoRA)	9–10, 36–37
RAG, vector databases, hybrid search, reranking, GraphRAG	16–23
Langfuse / LangSmith / ragas / observability	24–26
LangChain, LangGraph, LlamaIndex, Pydantic AI, CrewAI, MCP	20, 27–32
Reasoning models and multimodal models	33–34
PyTorch, Hugging Face, fine-tuning	2, 8, 36–37
Docker, Kubernetes, GitHub Actions, MLflow	38, 40
AWS Bedrock, Azure AI Foundry, GCP Vertex AI	39
Responsible AI, security, governance	22, 41
Production deployment and cost control	10, 14, 35, 37–40
Communication, solution design, interviews, career planning	42–45

Skills-to-class traceabilityTooling Standard for the Course

- **Languages:** Python 3.12, TypeScript (light use, for UI and Node integration demos)
- **Model building:** PyTorch, NumPy, tiktoken / Hugging Face tokenizers
- **Backend:** FastAPI, Pydantic, Pydantic AI, httpx, pytest
- **LLM providers:** OpenAI, Anthropic Claude, Google Gemini, plus open models via Hugging Face and Ollama
- **Retrieval:** pgvector (primary), Pinecone and Weaviate (comparison), Neo4j for graph retrieval, BM25 and cross-encoder rerankers
- **Orchestration:** LangChain, LangGraph, LlamaIndex, CrewAI (comparison), MCP

Class-by-Class Plan

Each row gives the **topic**, **what is covered**, and **the lab or deliverable** students finish in or right after class.

Module 1: Neural Networks, Tokens and Attention (Classes 1–6)

#	Topic	Content	Lab / Deliverable
1	The AI engineer role, AI terminology and the job market	AI vs ML vs deep learning vs GenAI vs agents; the vocabulary every later class uses (parameters, weights, loss, inference, embeddings, context window, hallucination); role map (AI vs ML vs MLOps vs Forward-Deployed); what Dice and LinkedIn postings ask for; course roadmap; dev environment (VS Code, uv, venv, Git)	Environment setup, repo created, first commit; personal glossary; annotate 3 real postings
2	Neural networks end to end with MNIST	Tensors, forward pass, loss functions, gradients and backpropagation derived step by step, optimizers, batching, overfitting and regularization, the PyTorch training loop, GPUs and Colab	Train an MNIST digit classifier, plot loss and accuracy, inspect misclassified digits and the confusion matrix
3	Production Python for AI: typing, Pydantic, async and HTTP	Type hints, dataclasses, Pydantic models, generators, asyncio, httpx, retries and backoff, rate limits, REST and JSON, secrets handling, packaging	Typed async API fetcher with retry and timeout logic
4	Tokenization and vectorization	Byte-pair encoding, vocabulary size and its cost impact, token counting, embeddings as vectors, cosine similarity and dot product derived with linear algebra, your first LLM API call	Build a mini BPE tokenizer; compute vector similarity between sentences; first LLM call with a token and cost calculator across 3 providers
5	The attention mechanism from scratch	Why recurrent models fell short, scaled dot-product attention, softmax, attention weights as a similarity lookup, positional information	Implement attention in PyTorch and visualize attention heatmaps for sample sentences
6	QKV matrices: self-, cross- and multi-head attention	Query, key and value projections, self-attention vs cross-attention, multi-head attention and head specialization, tensor shapes, quadratic cost in sequence length	Build a multi-head attention module; compare self-attention and cross-attention outputs on a translation-style task

Module 2: Building and Optimizing a Transformer (Classes 7–10)

#	Topic	Content	Lab / Deliverable
7	Causal masking and the decoder-only transformer	Causal mask, next-token prediction objective, residual connections, layer normalization, feed-forward blocks, positional encodings, stacking blocks	Add causal masking to your attention module and assemble a full decoder block; test that no future-token leakage occurs
8	LLM coding lab: build and train a Code GPT	Dataset preparation, training loop, learning-rate schedules, sampling with temperature, top-k and top-p, loss curves, text generation	Train a small GPT on a code or text corpus and generate samples under different sampling settings
9	Thinking like an engineer: how massive models are trained and reach production	Pretraining data pipelines and filtering, scaling laws, distributed training (data, tensor and pipeline parallelism), mixed precision, post-training (SFT, RLHF, DPO), evaluation before release, the path from checkpoint to API	Estimate compute, memory and cost for training a 7B-parameter model; read a model card; diagram a training-to-serving pipeline
10	Optimization hacks: KV caching, quantization and LoRA	Why generation is memory-bound, KV cache mechanics and memory cost, INT8/INT4 quantization and its accuracy trade-offs, the low-rank update behind LoRA derived, batching and latency	Add KV caching to your Code GPT and measure the speedup; quantize an open model and compare quality and memory; attach a LoRA adapter

Module 3: LLM Application Engineering (Classes 11–15)

#	Topic	Content	Lab / Deliverable
11	Prompt engineering and provider APIs	System vs user prompts, few-shot, reasoning prompts vs direct answers, delimiters, prompt versioning, OpenAI/Anthropic/Gemini SDKs, message formats, streaming, error handling, provider fallback, prompt-injection preview	Provider-agnostic client with fallback and a library of 5 versioned prompts compared side by side
12	Structured outputs	JSON mode, schema-constrained output, Pydantic validation, repair and retry loops, extraction patterns	Invoice/resume extractor returning validated objects
13	Tool use and function calling	Tool schemas, the calling loop, parallel calls, error handling, when the model should not call a tool	Assistant that calls weather, calculator and SQL tools
14	Cost, latency and reliability	Prompt caching, batching, model routing (small vs large), token budgets, caching layers, timeouts, circuit breakers	Add caching and routing to the client; benchmark cost and latency before and after
15	Serving an LLM service with FastAPI	Endpoints, streaming (SSE), auth, request validation, background tasks, pytest for LLM code (mocking)	**Deliverable:** streaming chat API with tests and a lightweight UI

Module 4: Retrieval-Augmented Generation (Classes 16–23)

#	Topic	Content	Lab / Deliverable
16	The RAG problem and architecture	What RAG solves (knowledge cutoff, private data, hallucination), RAG components end to end, embedding models, similarity metrics, when RAG beats fine-tuning	Build a naive RAG pipeline end to end and record where it fails
17	Document ingestion and chunking	PDF/HTML/DOCX parsing, OCR cases, chunking strategies (fixed, semantic, structure-aware), effect of chunk size and overlap, metadata design	Ingestion pipeline with 3 chunking strategies; compare retrieval quality
18	Vector databases and indexing	pgvector (primary), Pinecone and Weaviate comparison, indexes (HNSW, IVF), filters, similarity scoring, scaling and cost	Load the corpus into pgvector; run metadata-filtered similarity queries
19	Advanced retrieval: hybrid search and reranking	Why naive RAG hits accuracy ceilings, BM25 plus dense retrieval, reciprocal rank fusion, cross-encoder rerankers, top-k tuning	Add hybrid retrieval and a cross-encoder reranker; measure the recall improvement
20	Query rewriting, HyDE and RAG frameworks	Query rewriting, multi-query, HyDE, parent-document retrieval, citations and source attribution; LlamaIndex and LangChain compared with plain code	Build the same RAG three ways (raw, LlamaIndex, LangChain) with query rewriting added; compare complexity and quality
21	RAG architectures: GraphRAG, KAG, agentic and multimodal RAG	Common pitfalls in RAG, knowledge graphs with Neo4j and GraphRAG, knowledge-augmented generation (KAG), agentic RAG with retrieval planning, multimodal RAG over images and documents, choosing the right architecture for a use case	Prototype a GraphRAG or agentic RAG variant and compare it with your hybrid baseline on the same questions
22	RAG safety, guardrails and Code RAG	Prompt injection in retrieved content, PII redaction, access control and multi-tenant data, guardrail layers (input, output, retrieval), grounding checks, RAG over a code repository (code-aware chunking, symbol and dependency context)	Add guardrails to your RAG service; build a Code RAG assistant over a real repository
23	RAG project build day	Architecture review, freshness and re-indexing, failure analysis, deployment checklist	**Project 1:** Production RAG over a real corpus, deployed with citations and guardrails

Module 5: Evaluation and Observability (Classes 24–26)

#	Topic	Content	Lab / Deliverable
24	Evaluation design and RAG-specific evals	Why vibes fail, golden datasets, synthetic test data, retrieval metrics (recall@k, MRR), generation metrics (faithfulness, answer relevance, context precision), error taxonomies	50-question golden set for Project 1 with retrieval and generation scores
25	Evaluation tools and evals in CI	ragas, LLM-as-judge (bias and calibration), human review loops, pairwise comparison, eval gates in GitHub Actions, regression testing for prompt and model changes	Automated ragas run, judge calibration against human labels, and a CI pipeline that fails the build when scores drop
26	Tracing and observability	Langfuse and LangSmith, OpenTelemetry, spans for retrieval/LLM/tool steps, cost and latency dashboards, Grafana, online feedback capture	Fully traced Project 1 with a live dashboard

Module 6: Agents, Context Engineering and MCP (Classes 27–32)

#	Topic	Content	Lab / Deliverable
27	Single-agent systems: tool calling, ReAct and Pydantic AI	What an agent is, the function-calling loop, the ReAct pattern (reason, act, observe), plan-and-execute, Pydantic AI for typed agents, failure points of single agents (looping, tool misuse, compounding errors), when an agent is the wrong answer	Build a ReAct agent from scratch, then rebuild it with Pydantic AI; log and categorize its failures
28	LangChain and LangGraph	LangChain components and when to use them, LangGraph state machines, nodes and edges, conditional routing, checkpointing, human-in-the-loop interrupts, retries, streaming state	Research agent built as a LangGraph graph with an approval step before side-effecting actions
29	Context engineering: prompts, memory and context windows	Context engineering vs prompt engineering, what goes into the context window and in what order, short-term memory, long-term memory stores, summarization and compaction, retrieval over history, context for coding agents (repository instructions, scoped file context)	Add short- and long-term memory to the LangGraph agent; configure a coding agent with project context and measure the effect
30	Model Context Protocol (MCP)	MCP architecture (hosts, clients, servers), tools, resources and prompts, building an MCP server, connecting clients, authentication and security considerations, enterprise integration patterns	Custom MCP server exposing an internal-style API, consumed by your agent
31	Multi-agent systems	Agentic design patterns (supervisor/worker, handoff, critic, parallel fan-out), building agents with LangGraph and CrewAI, problems unique to multi-agent systems (orchestration, information isolation, planning, cost), managing memory across agents	Supervisor with 3 specialist agents sharing scoped memory
32	Agent evaluation, trade-offs and project day	Trajectory evals, tool-call accuracy, guardrails on actions, observability for agents, cost vs accuracy vs latency trade-offs, running a full eval suite on a multi-agent pipeline	**Project 2:** Multi-agent system with memory, MCP tools, human approval, traces and a full eval suite

Module 7: Reasoning, Multimodal and Model Customization (Classes 33–37)

#	Topic	Content	Lab / Deliverable
33	Thinking models: reasoning and chain of thought	Chain-of-thought prompting, how reasoning models are trained (reinforcement learning on verifiable rewards), test-time compute, reasoning budgets and cost, when thinking models help and when they only add latency	Compare a standard model with a reasoning model on math, planning and code tasks; build a router that picks between them
34	Multimodal models: images, video, CLIP and diffusion	Vision encoders, CLIP and contrastive training, vision-language models, video as frame sequences, diffusion models (forward noising, denoising, guidance), multimodal pipelines	Image search with CLIP embeddings; image question answering with a vision-language model; image generation with a diffusion pipeline
35	Prompting vs RAG vs fine-tuning: choosing with evidence	Decision framework for knowledge problems vs behavior problems, cost, accuracy and latency trade-offs, running the same eval across all three approaches, hybrid strategies	Solve one task three ways, then write a decision memo backed by eval scores and cost numbers
36	Fine-tuning in depth: SFT, LoRA, QLoRA and DPO	Dataset design and curation, supervised fine-tuning, LoRA and QLoRA hyperparameters and memory trade-offs, preference tuning with DPO, data contamination, before/after evaluation	Fine-tune a small open model on a domain dataset and benchmark it against the base model and a prompted frontier model
37	Inference serving and Project 3	vLLM, continuous batching, quantized serving, GPU cost and latency, Ray Serve, managed vs self-hosted decision	**Project 3:** Fine-tuned model served with vLLM behind your FastAPI service, with a cost analysis

Module 8: Cloud, MLOps and LLMOps (Classes 38–40)

#	Topic	Content	Lab / Deliverable
38	Docker and Kubernetes for AI services	Images, multi-stage builds, secrets, GPU containers, compose for app plus database, pods, deployments, services, autoscaling, when Kubernetes is overkill	Containerize Projects 1 and 2; deploy to a local cluster (kind or minikube) with autoscaling
39	AWS for AI engineers, with Azure and GCP compared	IAM, Bedrock (models, knowledge bases, guardrails, agents), S3, Lambda, ECS/Fargate, cost controls; Azure AI Foundry and Google Vertex AI equivalents; reading multi-cloud job requirements	Deploy your RAG service on AWS with Bedrock; port one component to Azure or GCP and fill in a comparison matrix
40	CI/CD, MLflow and LLMOps in production	GitHub Actions pipelines, environments, model and prompt registry, experiment tracking with MLflow, rollbacks, monitoring drift and quality, SLOs, cost governance, incident response	**Project 4:** Pipeline that tests, evaluates, builds and deploys on merge, with runbook, alert rules and a simulated incident

Module 9: Security and Governance (Class 41)

#	Topic	Content	Lab / Deliverable
41	AI application security, guardrails and governance	Direct and indirect prompt injection, data exfiltration, tool and permission abuse, PII handling, secrets, OWASP LLM Top 10, red-teaming; NIST AI RMF, ISO/IEC 42001, EU AI Act risk tiers, model cards, audit trails	Red-team your own RAG and agent, write a remediation list, and produce a model card and governance one-pager for your capstone

Module 10: Capstone and Career Launch (Classes 42–45)

#	Topic	Content	Lab / Deliverable
42	Capstone scoping, architecture review and engineering best practices	Capstone scoping, architecture reviews with peer and instructor critique, eval plan and risk register, AI engineering best practices (versioning, testing, cost budgets, rollback plans)	Approved capstone design document
43	Capstone build clinic	Live debugging, performance tuning, cost trimming, deployment help	Working capstone in staging
44	Capstone demos and AI system-design interviews	Live capstone demos with peer Q&A and structured feedback; designing RAG, agent and LLM platforms on a whiteboard, trade-off articulation, scaling, failure modes; customer-facing communication	Final demo, plus two timed mock system-design sessions
45	Career goals: moving into AI engineering	Role map, transition paths from your current IT role, resume keywords, LinkedIn headline and Featured project, GitHub portfolio, job-search plan with Dice and LinkedIn alerts, 12-month growth plan (detail in the Career Paths chapter)	Personal career plan, polished profile, 10 target roles shortlisted

Career Paths: Moving into AI Engineering from Any IT Field

AI engineering is software engineering plus a layer of AI-specific patterns (RAG, agents, evaluation) and tooling. System design, APIs, version control, testing and debugging transfer directly, so the new learning is the AI layer, not learning to program. One practitioner roadmap puts a structured transition at roughly 6 to 12 months, depending on where you start and how much you build.

AI engineering job roles

Role	What you do	Core skills	Classes
AI Engineer / Applied AI Engineer	Build and ship AI features inside products: RAG, agents, prompt and eval pipelines	Python, LLM APIs, RAG, evals, cloud	3–26, 38–40
LLM / GenAI Engineer	Build LLM systems: retrieval, generation pipelines, fine-tuning and model customization	PyTorch, Hugging Face, fine-tuning, vector databases	2–10, 16–23, 33–37
Agentic AI Engineer	Design agents and multi-agent workflows that use tools and memory	LangGraph, MCP, context engineering, agent evals	27–32
AI Platform / GenAI Infrastructure Engineer	Build the shared platform other teams use: gateways, model serving, observability, cost controls	Docker, Kubernetes, vLLM, cloud, tracing	10, 26, 37–40
MLOps / LLMOps Engineer	Deploy, monitor and version models and prompts; keep quality, latency and cost in range	CI/CD, MLflow, monitoring, cloud	25–26, 38–40
ML Engineer	Train and deploy traditional and deep-learning models	PyTorch, feature engineering, training pipelines	2, 8–9, 36
Data Scientist (GenAI)	Apply LLMs to analytics and business questions; run experiments and evaluations	Python, SQL, LLM APIs, evals	12, 24–25, 35
Forward-Deployed AI Engineer	Work with customers to design and deliver AI solutions; part engineer, part advisor	System design, communication, rapid prototyping	23, 32, 44
AI Architect / Solutions Architect	Design enterprise-scale AI systems; choose models, clouds and patterns; lead technical teams	Architecture, cloud, security, governance	21, 35, 39, 41
AI Product Engineer	Build AI-powered products end to end, combining front end, back end and AI	Full-stack, LLM APIs, UX for AI features	11–15, 23
AI Integration Specialist	Connect AI tools to existing business systems and workflows (enterprise copilots, automation)	APIs, MCP, enterprise platforms	13, 30
AI Evaluation / QA Engineer (emerging)	Design test sets, judges and regression gates for AI behavior	Evals, test automation, observability	24–26
AI Security Engineer (emerging)	Red-team and harden AI applications	Prompt-injection defense, guardrails, threat modeling	22, 41

Role	What you do	Core skills	Classes
AI Governance and Risk Specialist	Turn responsible-AI policy into risk assessments, audits and compliance evidence	NIST AI RMF, ISO/IEC 42001, EU AI Act	41
AI Research Engineer	Improve and build model architectures and run experiments	Deep learning, transformers, math	2, 5–10, 36

Role titles come from the Dice, LinkedIn and hiring-report sources we reviewed. The two roles marked emerging often appear under broader titles. The class mapping is our own judgment.

Transition map by current IT role

If you are a…	Target roles	What already transfers	What to add (classes)	Starter project
Backend / full-stack developer	AI Engineer, Applied AI Engineer, AI Product Engineer	APIs, system design, testing, databases	Transformer internals (4–8), RAG (16–23), agents (27–32), evals (24–26)	Production RAG with citations (Project 1)
Front-end / mobile developer	AI Product Engineer, AI Engineer	UX, state management, streaming interfaces	Python and FastAPI (3, 15), LLM APIs (11–14), RAG (16–23)	Streaming assistant with tool calling and a polished UI
Java / .NET enterprise developer	AI Engineer, AI Integration Specialist, AI Architect	Enterprise integration, security, scale	Python (3), LLM apps (11–15), MCP (30), cloud AI services (39)	Knowledge assistant over internal documents
Enterprise-platform consultant (SAP, Salesforce, ServiceNow)	AI Integration Specialist, Forward-Deployed AI Engineer, AI Architect	Business processes, integration, client delivery	Tool calling and MCP (13, 30), RAG (16–23), governance (41)	Copilot over an enterprise platform's documents and APIs
QA / test automation engineer	AI Evaluation Engineer, AI Engineer, LLMOps Engineer	Test design, regression thinking, automation	Evals (24–26), agent testing (32), CI gates (25, 40)	Eval suite with golden dataset and CI gate for an existing assistant
DevOps / SRE / cloud engineer	MLOps/LLMOps Engineer, AI Platform Engineer	CI/CD, containers, monitoring, cost control	Model serving (10, 37), LLM apps (11–15), RAG (16–23), observability (26)	Deployed, autoscaled model service with dashboards and runbook (Project 4)
Data analyst / BI developer	Data Scientist (GenAI), AI Engineer	SQL, metrics, storytelling with data	Python (3), SQL tool calling (13), RAG (16–23), evals (24–25)	Analyst copilot over SQL data and documents
Data engineer	AI Engineer, AI Platform Engineer, LLMOps Engineer	Pipelines, warehouses, orchestration	Chunking and vector databases (17–18), RAG (19–23), evals (24–26)	Ingestion and indexing pipeline with freshness handling
Data scientist / ML practitioner	LLM Engineer, ML Engineer, Applied AI Engineer	Modeling, statistics, experimentation	Production Python and serving (3, 15), RAG and agents (16–32), cloud and CI/CD (38–40)	Take a notebook RAG demo to a deployed, monitored service
IT support / system administrator	AI Integration Specialist, MLOps/LLMOps (entry)	Troubleshooting, scripting, tooling	Python (3), LLM apps (11–15), containers (38), agents (27–28)	IT helpdesk agent with approval step for actions
Business analyst / product or project manager	Forward-Deployed AI Engineer, AI Integration Specialist, AI Governance Specialist	Requirements, stakeholders, domain knowledge	Project 1 end to end, evals (24–25), trade-offs (35), governance (41)	Domain assistant with a decision memo on cost, accuracy and latency

If you are a…	Target roles	What already transfers	What to add (classes)	Starter project
Security / network engineer	AI Security Engineer, AI Governance Specialist	Threat modeling, access control	Guardrails (22), agent permissions (28, 30), security and governance (41)	Red-team report on your own RAG and agent
Database administrator	AI Engineer (data-heavy), AI Platform Engineer	Indexing, query tuning, backup and recovery	Vector databases (18), hybrid search (19), cloud (39)	Benchmark pgvector against a managed vector database on latency, recall and cost

Compensation snapshot

India, in lakh per annum (LPA) unless marked Cr (crore). Source: Instahyre hiring data, updated June 2026.

Level	AI Engineer	GenAI / LLM Engineer	MLOps Engineer	ML Engineer
Fresher (0–1 yr)	₹8–14	₹10–18	₹8–14	₹6–12
Early career (2–4 yr)	₹22–38	₹26–45	not published	₹16–30
Mid-level (5–7 yr)	₹38–65	₹45–75	₹28–50 (3–5 yr)	₹30–55
Senior (8–11 yr)	₹65 LPA–1.2 Cr	₹80 LPA–1.5 Cr	₹55–95	₹55–90
Staff / Principal (12+ yr)	₹1.1–1.8 Cr	₹1.4–2.5 Cr	₹1–1.8 Cr	₹95 LPA–1.5 Cr

- **United States:** one 2026 resume-guide comparison lists AI Engineer at $160K–$210K mid-level and $220K–$300K senior; ML Engineer $155K–$205K mid and $210K–$280K senior; MLOps Engineer $150K–$195K mid and $200K–$260K senior. Another analysis puts entry-level AI engineers at $120K–$160K.
-

Growth tracks after your first AI role

- **Individual contributor:** AI Engineer, Senior AI Engineer, Staff or Principal AI Engineer, owning larger systems and technical direction.
- **Specialist:** go deep on agents and context engineering, AI platform and inference, evaluation, security, or post-training and fine-tuning.
- **Architect:** move to AI Architect or Solutions Architect, choosing models, clouds and patterns across teams.
- **Customer-facing:** Forward-Deployed AI Engineer or AI solutions lead, combining delivery with advisory work.
- **Leadership:** engineering manager or head of an AI platform team, once you have shipped and operated several systems.